Project Design Documentation – Clover Line E-Commerce Platform & SmartFit Size Recommender

Last updated: September 2, 2025
Version: DRAFT
Date: 09/02/2025
Change Log: Updated with new 12-week plan and milestones.

---

1. Project Title & Version Control

Title: Clover Line – Secure E-Commerce Platform with SmartFit Size Recommender

Repository Strategy: Monorepo with separate frontend and backend folders.
Repo Structure:

```
cloverline/
├─ frontend/            # React + Vite + Tailwind
├─ backend/             # FastAPI + SQLAlchemy
├─ docs/                # Design docs, diagrams, ADRs, screenshots
├─ .github/workflows/   # CI pipelines (lint, test, build)
└─ README.md
```

Branching Model: trunk-based development with short-lived feature branches (e.g., feat/auth-jwt, fix/cart-precision).
Versioning: Semantic Versioning (SemVer) starting at v0.1.0 for MVP.

---

2. Project Summary

Clover Line is a full-stack e-commerce platform for a small clothing brand, featuring secure authentication, product catalog management, cart and checkout flows, order tracking, and admin dashboards. A built-in **SmartFit size recommender** improves conversions by suggesting

sizes based on body measurements or quick inputs like height and weight. The stack emphasizes security, performance, and user-friendly design.

---

3. Problem Statement / Use Case

Problem: Small apparel brands struggle with professional e-commerce solutions and accurate sizing, leading to lost sales and high return rates.

Use Cases:

- **Shopper:** Browse, view details, get size recommendations, add to cart, checkout, track orders.
- **Admin:** Manage products, users, orders, and monitor sales through a dashboard.
- **Brand Owner:** Needs secure, scalable, and cost-efficient deployment with easy maintenance.

---

4. Goals and Objectives

- **Security & Identity:** JWT auth, password hashing, secure cookies, HTTPS.
- **Reliable Commerce:** Stable cart/checkout/order system with persistent storage.
- **Smart Sizing UX:** Integrated SmartFit recommender (API + frontend).
- **Scalability & Deployment:** Smooth CI/CD workflows, PostgreSQL optimization, production hosting.

---

5. Key Features / Functions

- **Authentication:** Signup/login, JWT session management, user profiles.
- **Product Catalog:** CRUD for admins, search, filters, product details.
- **Cart & Checkout:** Cart persistence, simulated/real payments, order creation.
- **Orders:** Confirmation, status updates, admin management.
- **Wishlist:** Save/remove products, quick add-to-cart.
- **SmartFit Recommender:** Size suggestions with detailed or quick inputs.
- **Admin Dashboard:** Manage products, orders, users.
- **Email/Notifications:** Order confirmations, shipping updates.
- **Payments:** Stripe/PayPal integration with sandbox testing.
- **Deployment:** Hosting on Heroku/Vercel/Netlify, CI pipelines, monitoring.

---

6. Tech Stack and Tools

- **Frontend:** React, Vite, TailwindCSS, React Router.
- **Backend:** FastAPI (Python 3.11+), Pydantic, SQLAlchemy, Alembic.
- **Database:** PostgreSQL (prod), SQLite (dev).
- **Security:** JWT, bcrypt, input validation, HTTPS, secure cookies.
- **Payments:** Stripe/PayPal.
- **Testing:** pytest, Jest, React Testing Library.
- **CI/CD:** GitHub Actions, Pre-commit (black/ruff), Bandit, pip-audit.
- **Deployment:** Backend (Heroku/Render/Railway), Frontend (Vercel/Netlify).
- **Monitoring:** Structured logs, backups, error monitoring tools.

---

7. Architecture / Workflow Diagrams

7.1 High-Level Architecture

```
Client (React + Tailwind) → FastAPI API → PostgreSQL Database
                          → SMTP (emails)
                          → SmartFit recommender
                          → Stripe/PayPal (payments)
```

7.2 Auth & Product Flow (simplified sequence)

1. User signs up → backend stores hashed password.
2. Login → backend returns JWT.
3. JWT used for protected routes (products, cart, orders).

7.3 SmartFit Flow

1. Input (height/weight or full measurements).
2. Check available data.
3. Map to size chart → return suggestion + confidence.

---

8. Timeline / Weekly Milestones (New 12-Week Plan)

✅ Month 1 – Core Infrastructure (Days 1–16)

Week 1

- Day 1: GitHub Repo & README
- Day 2: Environment Setup (Python, Node.js, PostgreSQL)
- Day 3: Project Structure (frontend/backend folders)
- Day 4: Database Setup (users, products, orders)

Week 2

- Day 5: JWT Auth (register/login)
- Day 6: User Profile API (create, update, retrieve)
- Day 7: Frontend Auth UI (login/register forms)
- Day 8: Full Auth Flow Testing

Week 3

- Day 9: Product Schema & CRUD API
- Day 10: Product Catalog UI
- Day 11: Filters & Search
- Day 12: Product Detail Page

Week 4

- Day 13: Cart API (persist carts)
- Day 14: Cart UI (items, totals)
- Day 15: Checkout Backend (orders schema)
- Day 16: Checkout Frontend (order submission)

✅ Month 2 – Advanced Features (Days 17–32)

Week 5

- Day 17: Wishlist API
- Day 18: Wishlist UI
- Day 19: SmartFit Backend (size logic + API)
- Day 20: SmartFit Frontend (input form + results)

Week 6

- Day 21: Admin Dashboard Backend
- Day 22: Admin Dashboard Frontend
- Day 23: Order Management (status updates)

- Day 24: Notifications (frontend alerts)

Week 7

- Day 25: Payment Research & Setup (Stripe/PayPal)
- Day 26: Payment Backend
- Day 27: Payment Frontend (checkout component)
- Day 28: Payment Testing (test cards, fail states)

Week 8

- Day 29: Security Enhancements
- Day 30: Error Handling (global handlers + UI)
- Day 31: Unit Testing (frontend + backend)
- Day 32: Integration Testing (end-to-end workflows)

✅ Month 3 – Polish & Deployment (Days 33–48)

Week 9

- Day 33: UI/UX Improvements
- Day 34: Accessibility Updates
- Day 35: Performance Optimization
- Day 36: Database Optimization

Week 10

- Day 37: API Documentation
- Day 38: User Guide (shopping + SmartFit help)
- Day 39: Deployment Setup (hosting, env vars)
- Day 40: Backend Deployment

Week 11

- Day 41: Frontend Deployment
- Day 42: Domain & SSL Setup
- Day 43: Final Debugging (payments + SmartFit live)
- Day 44: Backup & Monitoring Setup

Week 12

- Day 45: Final Presentation Prep (demo + screenshots)
- Day 46: Portfolio Integration (GitHub Pages showcase)
- Day 47: Resume & LinkedIn Update

- Day 48: Project Wrap-Up (code, docs, deployment finalized)